

Informe de Investigación: Metodologías Modernas de Enseñanza de Programación y Requisitos para Emprendedores Deep-Tech en Contraste con el Sistema PUCP

Fecha de Elaboración: 20 de marzo de 2026

Resumen Ejecutivo

Este informe presenta una investigación exhaustiva sobre las metodologías pedagógicas de vanguardia en la enseñanza de la programación y las competencias críticas para el emprendimiento en tecnologías profundas (deep-tech), contrastando estos hallazgos con un análisis documentado del sistema de enseñanza de la Pontificia Universidad Católica del Perú (PUCP). La investigación revela una divergencia fundamental entre las prácticas modernas, adoptadas por universidades de élite y demandadas por la industria, y el modelo tradicional evidenciado en la PUCP. Mientras que la educación moderna en programación se orienta hacia el aprendizaje activo, la resolución de problemas complejos, el uso de herramientas de inteligencia artificial y la evaluación basada en proyectos que simulan el entorno profesional, el sistema analizado en la PUCP se caracteriza por una rigidez metodológica, un enfoque en la mecanización y la memorización, la imposición de restricciones artificiales y un sistema de evaluación punitivo centrado en la conformidad formal por encima de la comprensión conceptual.

El análisis comparativo con instituciones como MIT, Stanford y Carnegie Mellon University (CMU) demuestra que estas fomentan la autonomía, el pensamiento crítico y la aplicación práctica en contextos realistas. En contraste, el modelo de la PUCP, según la documentación provista, parece estar desconectado de las prácticas actuales de la industria, prohibiendo herramientas estándar como la IA generativa y el control de versiones, y enfocándose en aspectos sintácticos obsoletos. Adicionalmente, se exploran las competencias clave para los fundadores de startups deep-tech, destacando la necesidad de una combinación de pericia técnica, visión de negocio, resiliencia y habilidades de colaboración, características que el entorno académico tradicional, con su alta presión y aversión al riesgo, parece inhibir en lugar de cultivar. El informe concluye que el sistema documentado de la PUCP no solo presenta una obsolescencia pedagógica y tecnológica, sino que también podría estar contribuyendo a un entorno que merma la salud mental y el potencial emprendedor de sus estudiantes, presentando una propuesta de valor económica y profesional cada vez más cuestionable en el panorama tecnológico de 2026.

1. Introducción y Objetivo de la Investigación

El sector tecnológico evoluciona a una velocidad sin precedentes, impulsado por avances en inteligencia artificial, computación cuántica y biotecnología. En este contexto, el emprendimiento en tecnologías profundas (deep-tech) emerge como un motor clave para la innovación disruptiva y la resolución de los problemas más complejos de la humanidad. El éxito de estos emprendimientos depende críticamente del capital humano que los impulsa: individuos con una sólida formación técnica, una mentalidad innovadora y una capacidad excepcional para navegar la incertidumbre. Esto sitúa a las instituciones de educación superior en una posición crucial, como forjadoras de la próxima generación de líderes tecnológicos.

Sin embargo, existe un debate creciente sobre si los modelos educativos tradicionales, diseñados en una era industrial, están adecuadamente equipados para preparar a los estudiantes para los desafíos del siglo XXI. Las críticas académicas señalan que los enfoques basados en la memorización, la repetición y la evaluación estandarizada pueden sofocar la creatividad, el pensamiento crítico y la resiliencia, habilidades indispensables para la innovación.

El objetivo de este informe es realizar una investigación exhaustiva y académicamente rigurosa sobre las metodologías modernas de enseñanza de la programación y los requisitos para emprendedores deep-tech, para contrastar objetivamente estos hallazgos con el sistema de enseñanza de la Pontificia Universidad Católica del Perú (PUCP), según la documentación analizada. La investigación busca responder a preguntas fundamentales: ¿Cómo están evolucionando las pedagogías en ciencias de la computación? ¿Qué habilidades y competencias distinguen a los fundadores de startups tecnológicas exitosas? ¿De qué manera los sistemas de evaluación modernos reflejan las prácticas de la industria? Y, crucialmente, ¿cómo se compara un sistema universitario específico, como el de la PUCP, con estos estándares modernos?

Para lograr este objetivo, el informe se estructura en varias secciones clave. Primero, se examinan las metodologías pedagógicas actuales en programación, incluyendo el aprendizaje activo, el aprendizaje basado en problemas y la integración de la inteligencia artificial. Segundo, se analiza el enfoque de enseñanza de universidades de élite como MIT, Stanford y CMU. Tercero, se consolidan las críticas académicas al modelo tradicional. Cuarto, se definen las habilidades críticas para emprendedores deep-tech. Quinto, se discuten los sistemas de evaluación modernos. Finalmente, todos estos elementos se utilizan como marco de referencia para realizar un análisis crítico y detallado del sistema PUCP, basándose en la evidencia documental proporcionada, abarcando desde sus sílabos y métodos de evaluación hasta su impacto sistémico en la salud mental y la viabilidad económica para los estudiantes.

2. Metodologías Pedagógicas Modernas en la Enseñanza de la Programación (2020-2026)

La enseñanza de la programación ha experimentado una transformación significativa, alejándose de los modelos pasivos de transmisión de información hacia enfoques centrados en el estudiante que promueven la participación activa, la resolución de problemas y la construcción de conocimiento. Estas metodologías modernas buscan no solo enseñar la sintaxis de un lenguaje, sino también desarrollar el pensamiento computacional, la creatividad y las habilidades de colaboración necesarias en el entorno profesional actual.

2.1 Aprendizaje Activo y Aula Invertida (Flipped Classroom)

El aprendizaje activo engloba una variedad de estrategias que involucran a los estudiantes directamente en el proceso de aprendizaje. Una de sus implementaciones más populares en la educación superior, especialmente en ciencias de la computación e ingeniería, es el modelo de aula invertida o Flipped Classroom. Este enfoque invierte la estructura tradicional de la clase: la instrucción directa, como las clases teóricas, se traslada fuera del aula, generalmente a través de videos, lecturas o podcasts que los estudiantes revisan de forma autónoma antes de la sesión presencial. El tiempo en el aula, antes dedicado a la exposición del profesor, se libera para actividades de aprendizaje activo como la resolución de problemas, discusiones en grupo, debates y proyectos colaborativos.

Estudios han demostrado que esta metodología puede generar un beneficio significativo en el rendimiento promedio de los estudiantes en cursos de ciencias y mejorar la comprensión del contenido en carreras de ingeniería. Al fomentar un clima de trabajo más dinámico y participativo, el aula invertida

aumenta la motivación de los estudiantes y las interacciones sociales entre pares. El rol del estudiante se transforma, pasando de ser un receptor pasivo a un gestor activo de su propio aprendizaje, mientras que el profesor asume el papel de facilitador o guía, orientando a los estudiantes en la aplicación del conocimiento. La implementación exitosa de este modelo, sin embargo, requiere una cuidadosa planificación y una redefinición de la infraestructura tecnológica y las estrategias de enseñanza, asegurando que los estudiantes se comprometan con los materiales previos a la clase para poder participar activamente en las actividades presenciales.

2.2 Aprendizaje Basado en Problemas y Proyectos (PBL/PjBL)

El Aprendizaje Basado en Problemas (PBL, por sus siglas en inglés) es un método de enseñanza-aprendizaje arraigado en la teoría constructivista, que postula que el aprendizaje es un proceso activo y constructivo. En el PBL, el aprendizaje se organiza en torno a la investigación, explicación y resolución de un problema significativo y complejo, a menudo extraído de situaciones reales o simuladas del ámbito profesional. Los estudiantes, trabajando en pequeños grupos, son los protagonistas de su aprendizaje: identifican lo que necesitan saber para abordar el problema, investigan, aplican el conocimiento adquirido y proponen soluciones. El profesor actúa como un facilitador, guiando el proceso sin proporcionar respuestas directas.

Estrechamente relacionado, el Aprendizaje Basado en Proyectos (PjBL) comparte la filosofía centrada en el estudiante, pero se enfoca en la creación de un producto o artefacto tangible como culminación de un proceso de investigación y desarrollo extendido en el tiempo. Ambas metodologías son particularmente efectivas en programación, ya que el objetivo principal de la disciplina es, en esencia, la resolución de problemas. Estos enfoques conectan los conceptos teóricos con la aplicación práctica, aumentando la motivación de los estudiantes al trabajar en desafíos auténticos. Fomentan el desarrollo de habilidades transversales cruciales como el trabajo en equipo, la comunicación, la gestión del tiempo y el pensamiento crítico. Los errores se consideran oportunidades de aprendizaje, y el proceso de reconstrucción colectiva del conocimiento en grupo fortalece la comprensión conceptual.

2.3 El Rol de la Inteligencia Artificial en la Educación de la Programación

La integración de herramientas de inteligencia artificial (IA), como GitHub Copilot, en la enseñanza universitaria de la programación representa una de las evoluciones pedagógicas más recientes y disruptivas. Estas herramientas, entrenadas con miles de millones de líneas de código, actúan como “programadores de pareja” de IA, ofreciendo sugerencias de código en tiempo real, desde líneas individuales hasta funciones completas. Su adopción en el aula busca preparar a los estudiantes para un panorama industrial donde el desarrollo asistido por IA se está convirtiendo en el estándar.

Los beneficios pedagógicos son múltiples. La IA puede aumentar significativamente la productividad de los estudiantes, permitiéndoles enfocarse en la lógica de alto nivel y los aspectos más complejos de un proyecto en lugar de en tareas repetitivas o sintaxis básica. Actúa como una herramienta de aprendizaje continuo, ya que al observar y analizar las sugerencias de la IA, los estudiantes pueden descubrir nuevas formas de abordar un problema y aprender buenas prácticas de programación. Además, el uso de estas herramientas en proyectos académicos proporciona una experiencia práctica y realista, otorgando a los estudiantes una ventaja competitiva en el mercado laboral.

No obstante, la comunidad académica subraya la importancia crítica de un enfoque pedagógico equilibrado. Existe una preocupación legítima de que una dependencia excesiva de estas herramientas pueda atrofiar el desarrollo de habilidades fundamentales de resolución de problemas y pensamiento crítico. Por lo tanto, es imperativo que los estudiantes primero adquieran una base sólida en los fundamentos de la programación. Solo con una comprensión contextual profunda pueden evaluar eficazmente las sugerencias de la IA, depurar el código generado y aplicar su propia creatividad para

resolver problemas complejos. La recomendación general es promover un uso supervisado y responsable, combinando proyectos complejos donde se permite el uso de IA con ejercicios fundamentales diseñados para fortalecer la autonomía y el pensamiento crítico del estudiante.

3. Análisis Comparativo: La Enseñanza de Programación en Universidades de Élite

Las universidades de mayor prestigio mundial en ciencias de la computación, como el Instituto Tecnológico de Massachusetts (MIT), la Universidad de Stanford y la Universidad Carnegie Mellon (CMU), han desarrollado enfoques pedagógicos que, si bien distintos en sus matices, comparten principios fundamentales orientados a la profundidad conceptual, la aplicación práctica y la preparación para los desafíos del mundo real.

3.1 MIT: Fundamentos y Algoritmos

El enfoque del MIT, visible en cursos como 6.0001 Introduction to Computer Science and Programming in Python y 6.006 Introduction to Algorithms, se centra en construir una base sólida y rigurosa desde el principio. El curso introductorio 6.0001 está diseñado para estudiantes sin experiencia previa y tiene como objetivo principal ayudarles a comprender el pensamiento computacional y la resolución de problemas. Utiliza Python como vehículo para enseñar conceptos fundamentales, pero el énfasis no está en el lenguaje en sí, sino en la capacidad de escribir programas para realizar tareas útiles.

Una vez sentadas las bases, el currículo avanza rápidamente hacia la profundidad teórica con cursos como 6.006, que introduce el modelado matemático de problemas computacionales. Este curso cubre algoritmos comunes, paradigmas algorítmicos y estructuras de datos, enfatizando la relación intrínseca entre los algoritmos y la programación. Se introducen técnicas de análisis de rendimiento, enseñando a los estudiantes no solo a resolver un problema, sino a hacerlo de manera eficiente. Los trabajos prácticos, como el desarrollo de algoritmos para resolver un Cubo de Rubik, demuestran el enfoque del MIT en aplicar la teoría algorítmica a problemas complejos y tangibles. La disponibilidad de materiales de curso completos a través de MIT OpenCourseWare (OCW), incluyendo videos de clases, notas y tareas, subraya un compromiso con la accesibilidad y la transparencia del conocimiento.

3.2 Stanford: Metodología, Ética y Aplicaciones Prácticas

La Universidad de Stanford, a través de su emblemático curso CS106A: Programming Methodology, adopta un enfoque que equilibra el rigor técnico con una fuerte orientación hacia la aplicación práctica y la conciencia ética. El curso está diseñado para ser una introducción amplia y atractiva a la programación, utilizando Python para explorar una variedad de dominios, desde el procesamiento de imágenes y la criptografía hasta el análisis de datos, el dibujo y la inteligencia artificial. La estructura de las tareas, que progresan desde ejercicios de calentamiento hasta proyectos más complejos como “Sand” o “BabyGraphics”, refleja una filosofía de “aprender haciendo”.

Un aspecto distintivo del enfoque de Stanford es la integración explícita de la ética en el currículo. Módulos como “Encryption Ethics” y la existencia de una “Política de Retracción y Citación Retroactiva” para violaciones del código de honor indican que la universidad no solo se preocupa por formar programadores competentes, sino también ciudadanos tecnológicos responsables. El lema del curso, “Everyone is welcome. Be kind. Be humane. Meet someone new. Learn by doing. Adapt to new contexts”, encapsula una cultura de inclusión, colaboración y aprendizaje continuo. La disponibilidad de múltiples recursos de apoyo, como foros de discusión, horas de ayuda y guías de estilo, fomenta un entorno de aprendizaje colaborativo y bien respaldado.

3.3 Carnegie Mellon University (CMU): Rigor en los Principios de la Computación

Carnegie Mellon University es reconocida por su enfoque riguroso y basado en principios en la enseñanza de las ciencias de la computación. El curso 15-122: Principles of Imperative Computation es un claro ejemplo de esta filosofía. Desde el título mismo, se establece un enfoque en los “principios” subyacentes de la computación imperativa, más allá de la sintaxis de un lenguaje específico (aunque utiliza un subconjunto seguro de C llamado C0). El curso está meticulosamente estructurado con una combinación de clases teóricas, sesiones de práctica (“precepts”) y evaluaciones frecuentes (“checkins”).

El calendario del curso revela un plan de estudios altamente organizado y progresivo, con problemas de práctica y tareas de programación semanales que construyen sistemáticamente la comprensión del estudiante. El peso asignado a cada componente de la evaluación y las fechas claras de entrega y corrección reflejan un sistema transparente y predecible. Las reglas para las horas de consulta, que estipulan que son para ayuda conceptual y para estudiantes que ya han realizado un esfuerzo independiente, fomentan la autonomía y la preparación. Este enfoque de CMU busca inculcar una comprensión profunda y disciplinada de cómo funcionan las computadoras a un nivel fundamental, preparando a los estudiantes para razonar sobre la corrección y eficiencia de los programas con una precisión casi matemática.

3.4 Síntesis del Enfoque de las Universidades de Élite

A pesar de sus diferencias estilísticas, los enfoques de MIT, Stanford y CMU convergen en varios principios clave que definen la enseñanza moderna de la programación. Primero, todas priorizan la comprensión conceptual y los principios fundamentales sobre la memorización de la sintaxis de un lenguaje particular. Segundo, utilizan proyectos y tareas prácticas que son a la vez desafiantes y relevantes, permitiendo a los estudiantes aplicar la teoría a problemas concretos. Tercero, fomentan un ecosistema de aprendizaje rico en recursos y apoyo, promoviendo la colaboración y la autonomía del estudiante. Cuarto, aunque no siempre de forma explícita en los sílabos introductorios, sus programas avanzados integran prácticas de la industria como el control de versiones, las pruebas y el diseño de software a gran escala. Finalmente, instituciones como Stanford están liderando el camino en la integración de consideraciones éticas directamente en el tejido de la educación técnica, reconociendo la profunda responsabilidad social de los tecnólogos.

4. Críticas Académicas al Modelo de Enseñanza Tradicional

El modelo de enseñanza tradicional, a menudo caracterizado por clases magistrales, aprendizaje pasivo y un fuerte énfasis en la memorización y la evaluación sumativa, ha sido objeto de intensas críticas por parte de teóricos de la educación y psicólogos durante décadas. Estas críticas son particularmente pertinentes en el campo de la programación, una disciplina inherentemente creativa y orientada a la resolución de problemas.

4.1 Limitaciones del Aprendizaje Pasivo y la Memorización

Una de las críticas más fundamentales al modelo tradicional proviene del conductismo, notablemente de B.F. Skinner. Skinner argumentaba que la educación tradicional era en gran medida ineficaz porque se basaba excesivamente en el control aversivo (el miedo a las malas notas, la crítica o el ridículo) en lugar del refuerzo positivo. Sostenía que el refuerzo, para ser efectivo, debe ser inmediato, algo que rara vez ocurre en un sistema donde la retroalimentación se da días o semanas después de una evaluación. La “enseñanza programada” de Skinner proponía una alternativa basada en la participación

activa del estudiante, la presentación del material en pequeños pasos de dificultad gradual y la retroalimentación inmediata, principios que resuenan en las metodologías de aprendizaje activo actuales.

Desde una perspectiva cognitiva, el énfasis excesivo en la memorización sin una reflexión crítica concurrente es ampliamente criticado por conducir a un aprendizaje superficial. Cuando los estudiantes son evaluados principalmente por su capacidad para recordar y reproducir información, pueden desarrollar una comprensión frágil y desconectada de la aplicación práctica. Este tipo de conocimiento es a menudo olvidado rápidamente después del examen porque no se ha integrado en una red conceptual significativa. La pedagogía moderna aboga por un equilibrio donde la memorización de conceptos fundamentales sirva como base para la reflexión, el análisis, la interpretación y la aplicación, llevando a un aprendizaje significativo y duradero. En programación, esto se traduce en la diferencia entre memorizar la sintaxis de un bucle y comprender conceptualmente la iteración y saber cuándo y cómo aplicarla para resolver un problema.

4.2 El Impacto de las Restricciones Artificiales en el Aprendizaje

Una práctica común en la enseñanza tradicional de la programación es la imposición de restricciones artificiales, como la prohibición de usar ciertas herramientas, bibliotecas o recursos en línea durante las evaluaciones o proyectos. Si bien la intención puede ser asegurar que los estudiantes dominen los fundamentos desde cero, la investigación y la crítica pedagógica sugieren que esta práctica puede ser contraproducente. Prohibir herramientas y bibliotecas que son estándar en la industria aísla a los estudiantes del ecosistema profesional real y les impide desarrollar una habilidad crucial: la capacidad de aprovechar el trabajo existente para construir soluciones más complejas y eficientes.

Esta prohibición puede sofocar la creatividad y el espíritu investigativo. En lugar de aprender a buscar, evaluar e integrar soluciones y componentes existentes, se enseña a los estudiantes a “reinventar la rueda”, una práctica ineficiente y poco común en el desarrollo de software moderno. Además, la prohibición de acceso a recursos como la documentación en línea o foros comunitarios durante el aprendizaje y la evaluación simula un entorno de trabajo irrealista y priva a los estudiantes de la oportunidad de desarrollar habilidades de búsqueda de información y depuración autónoma, que son vitales para cualquier programador profesional. Estas restricciones pueden crear una brecha entre la formación académica y las competencias demandadas por el mercado laboral, y pueden exacerbar las desigualdades para estudiantes con menos acceso previo a la tecnología.

4.3 Mecanización vs. Comprensión Conceptual Profunda

El modelo tradicional a menudo promueve la mecanización del aprendizaje, donde los estudiantes aprenden a seguir una serie de pasos prescritos para llegar a una única “respuesta correcta”. Este enfoque, si bien puede ser eficaz para resolver problemas estandarizados y repetitivos, falla en desarrollar el pensamiento crítico, la flexibilidad cognitiva y la capacidad de resolver problemas novedosos y ambiguos. En la programación, esto se manifiesta cuando la evaluación se centra en la conformidad con un formato de salida exacto, el uso de un algoritmo específico o la adherencia a convenciones de nomenclatura arbitrarias, en lugar de la elegancia, eficiencia o robustez de la solución.

Este énfasis en la mecanización puede llevar a los estudiantes a adoptar un rol pasivo, viéndose a sí mismos como meros ejecutores de instrucciones en lugar de diseñadores y arquitectos de soluciones. Se desalienta el cuestionamiento, la experimentación y la exploración de enfoques alternativos. El resultado es una comprensión superficial, donde el estudiante puede ser capaz de reproducir una solución para un problema conocido, pero es incapaz de adaptar su conocimiento a un nuevo contexto o de diseñar una solución para un problema que no ha visto antes. La educación moderna, en cambio, busca fomentar la comprensión conceptual profunda, donde los estudiantes no solo saben “qué” hacer, sino que entienden “por qué” lo hacen, lo que les permite transferir y aplicar su conocimiento de manera flexible y creativa.

5. Competencias Críticas para Emprendedores Deep-Tech (2025-2026)

El emprendimiento en tecnologías profundas (deep-tech) se distingue por su base en descubrimientos científicos significativos o innovaciones de ingeniería disruptivas. Estas empresas abordan problemas complejos, a menudo con largos ciclos de desarrollo y una alta necesidad de capital. Para tener éxito en este exigente campo en el horizonte 2025-2026, los fundadores necesitan un conjunto de competencias multifacéticas que van mucho más allá de la pericia técnica.

5.1 Competencias Técnicas y Científicas

La base de cualquier empresa deep-tech es una ventaja tecnológica defendible. Por lo tanto, es indispensable que los fundadores posean un conocimiento técnico y científico profundo en su dominio. Para el período 2025-2026, las áreas tecnológicas clave incluyen agentes de inteligencia artificial avanzados con capacidades de razonamiento sofisticado, la computación cuántica (especialmente en sus aplicaciones a la ciberseguridad), la convergencia de la IA con las ciencias de la vida para el descubrimiento de fármacos, y el desarrollo de nuevos materiales y robótica. El concepto de “Bits to Atoms”, que conecta el software con aplicaciones físicas en energía, movilidad y manufactura, será un campo fértil para la innovación. Los emprendedores deep-tech no solo deben comprender estas tecnologías, sino también ser capaces de liderar su desarrollo, navegar por la propiedad intelectual y traducir la investigación de laboratorio en productos viables.

5.2 Habilidades de Negocio y Gestión

La excelencia técnica por sí sola es insuficiente. Los fundadores deep-tech deben ser capaces de tender un puente entre el laboratorio y el mercado. Esto requiere un conjunto sólido de habilidades de negocio, incluyendo la validación temprana del mercado para identificar nichos con una necesidad real, el desarrollo de modelos de negocio sostenibles y la capacidad de escalar un prototipo a un producto comercial. La planificación financiera a largo plazo es particularmente crítica, dado que las empresas deep-tech a menudo enfrentan un “valle de la muerte” de financiación más largo que las startups de software tradicionales. La capacidad de asegurar rondas de financiación, obtener subvenciones y gestionar un presupuesto a lo largo de ciclos de desarrollo de varios años es fundamental. Además, la formación de alianzas estratégicas con universidades, centros de investigación y corporaciones establecidas es clave para acelerar el desarrollo tecnológico y acceder a canales de mercado.

5.3 Habilidades Blandas (Soft Skills) y Mentalidad Emprendedora

En un entorno tan complejo y multidisciplinario, las habilidades blandas se vuelven tan importantes como las competencias técnicas. La **comunicación efectiva** es primordial para articular una visión técnica compleja a inversores, clientes y empleados no técnicos. El **liderazgo** y la **empatía** son cruciales para construir y motivar equipos de alto rendimiento compuestos por científicos, ingenieros y profesionales de negocios. La **adaptabilidad** y la capacidad de **aprendizaje continuo** son vitales en un campo donde la tecnología evoluciona rápidamente.

El **pensamiento crítico** es especialmente importante en la era de la IA, ya que los fundadores deben ser capaces de cuestionar, validar e interpretar los resultados de los modelos algorítmicos. La **resolución de problemas complejos** y la **resiliencia** son quizás las características más definitorias; el camino del deep-tech está plagado de fracasos técnicos y de mercado, y solo aquellos con una alta tolerancia al riesgo y una capacidad para recuperarse de los contratiempos pueden perseverar. La **creatividad** para proponer soluciones novedosas y la **inteligencia emocional** para gestionar la presión y las relaciones interpersonales completan el perfil del emprendedor deep-tech exitoso.

5.4 El Perfil del Fundador Exitoso: Lecciones de Aceleradoras como Y Combinator

Las aceleradoras de startups de élite como Y Combinator (YC) ofrecen una visión valiosa de las cualidades que predicen el éxito empresarial. La filosofía de YC, encapsulada en el consejo de su fundador Paul Graham de “construir algo que la gente quiera”, se centra en la primacía del fundador. YC busca individuos “excepcionales”, descritos como implacables, obsesivos y resilientes. Hay un fuerte énfasis en los fundadores con una sólida formación técnica, ya que se considera más fácil enseñar a un ingeniero a vender que a un vendedor a codificar.

La capacidad de comunicar una idea de forma clara y concisa es altamente valorada, ya que es fundamental para atraer talento, capital y clientes. YC también prioriza a los equipos que ya han logrado cierta tracción inicial, ya sea en forma de usuarios o ingresos, como validación de que están resolviendo un problema real. La ambición de abordar mercados masivos y el potencial de crecimiento exponencial (“unicornio”) son expectativas clave. En esencia, el perfil del fundador de YC se alinea estrechamente con los requisitos del emprendedor deep-tech: una combinación de destreza técnica, una profunda comprensión del problema a resolver, habilidades de comunicación y una mentalidad de crecimiento y resiliencia inquebrantables.

6. Sistemas de Evaluación Modernos en Ingeniería de Software

La evaluación en la educación superior de ingeniería de software está transitando desde los exámenes tradicionales, que a menudo miden la memorización de conceptos teóricos, hacia métodos más auténticos que evalúan la capacidad de los estudiantes para aplicar sus conocimientos en la práctica. Estos sistemas modernos buscan reflejar los procesos y estándares de la industria del software, proporcionando una medida más holística y realista de la competencia de un estudiante.

6.1 Evaluación Basada en Proyectos y Portafolios

La evaluación basada en proyectos es una piedra angular de la educación moderna en ingeniería de software. En lugar de resolver problemas pequeños y aislados, los estudiantes se involucran en el desarrollo de un proyecto de software a lo largo de un semestre o incluso un año. Este enfoque permite una evaluación continua que abarca todo el ciclo de vida del desarrollo de software, desde la concepción y el análisis de requisitos, pasando por el diseño y la implementación, hasta las pruebas y el despliegue. Los proyectos a menudo simulan escenarios del mundo real, requiriendo que los estudiantes tomen decisiones de diseño, gestionen la complejidad y trabajen en equipo. La evaluación no se limita al código final, sino que también puede incluir la calidad de la documentación, la arquitectura del sistema y la gestión del proyecto.

Complementando los proyectos, los portafolios de software se están convirtiendo en una herramienta de evaluación cada vez más popular. Un portafolio es una colección curada del trabajo de un estudiante que demuestra su crecimiento, habilidades y logros a lo largo del tiempo. Puede incluir muestras de código, documentación de proyectos, contribuciones a proyectos de código abierto, ensayos reflexivos sobre su proceso de aprendizaje y certificaciones. Los portafolios ofrecen una visión mucho más rica y completa de las capacidades de un estudiante que una simple calificación, y sirven como un activo valioso para la búsqueda de empleo, permitiendo a los empleadores potenciales ver evidencia tangible de sus habilidades.

6.2 Revisiones de Código (Code Reviews) y Feedback Continuo

Inspirado directamente en una de las prácticas más importantes de la industria del software, el uso de revisiones de código como herramienta de evaluación y aprendizaje está ganando terreno. En este

proceso, el código de un estudiante es revisado por sus compañeros o por el instructor, quienes proporcionan retroalimentación constructiva sobre aspectos como la corrección, la eficiencia, la legibilidad, la adherencia a los estándares de codificación y la calidad del diseño. Este método tiene un doble beneficio pedagógico: los estudiantes aprenden a recibir y a incorporar críticas constructivas para mejorar su propio trabajo, y también desarrollan sus habilidades de análisis crítico al revisar el código de otros.

Las revisiones de código fomentan una cultura de colaboración y responsabilidad compartida por la calidad del software. Mueven la evaluación de un evento sumativo único al final de un proyecto a un proceso formativo y continuo. La retroalimentación constante permite a los estudiantes iterar y mejorar su trabajo, lo que conduce a un aprendizaje más profundo y a un producto final de mayor calidad. Al participar en este proceso, los estudiantes no solo mejoran sus habilidades técnicas, sino que también desarrollan habilidades de comunicación profesional y trabajo en equipo, preparándolos de manera más efectiva para el entorno colaborativo del desarrollo de software moderno.

7. Análisis Crítico del Sistema PUCP: Un Contraste Documentado

El análisis de la documentación proporcionada sobre el sistema de enseñanza de programación en la Pontificia Universidad Católica del Perú (PUCP) revela un profundo contraste con las metodologías modernas, los enfoques de las universidades de élite y las competencias requeridas por la industria tecnológica y el emprendimiento deep-tech. El modelo que emerge de los sílabos y las evaluaciones de laboratorio parece estar anclado en un paradigma tradicional, caracterizado por la rigidez, la obsolescencia y un enfoque punitivo.

7.1 Metodologías Pedagógicas y Estructura Curricular

La metodología descrita en los sílabos de cursos como INF144 (Técnicas de Programación) y 1INF25 (Programación 2) se basa en “clases expositivas con intervención de los estudiantes” y “laboratorios calificados y dirigidos”. Este modelo de transmisión unidireccional de conocimiento, complementado con evaluaciones estandarizadas, se aleja de los enfoques de aprendizaje activo, invertido y basado en problemas que caracterizan a la educación moderna. Los objetivos de aprendizaje declarados en INF144, por ejemplo, ponen un fuerte énfasis en la forma (alineación, nomenclatura, comentarios) más que en el contenido (resolución efectiva de problemas, creatividad algorítmica).

La estructura curricular muestra un enfoque desproporcionado en temas que han perdido centralidad en el desarrollo de software moderno. En 1INF25, el capítulo sobre “Arreglos y Punteros”, que se centra en la gestión manual de la memoria (aritmética de punteros, punteros a punteros), ocupa 16 horas, casi el 30% del curso. Esto contrasta con la práctica moderna en C++, que favorece el uso de punteros inteligentes y el principio RAII (Resource Acquisition Is Initialization) para una gestión de memoria más segura y automática. La enseñanza intensiva de la gestión manual de memoria es análoga a enseñar la mecánica de un carburador en la era de la inyección electrónica: es un conocimiento de nicho, propenso a errores y en gran medida obsoleto. Mientras tanto, temas cruciales como el control de versiones (Git), las pruebas de software, los patrones de diseño, las arquitecturas de software y el consumo de APIs están notablemente ausentes de los sílabos analizados.

7.2 El Sistema de Evaluación: Rigidez, Penalización y Restricciones

El sistema de evaluación de los laboratorios es quizás la manifestación más clara de la rigidez del modelo. Los diez laboratorios analizados siguen una estructura casi idéntica, repetitiva y altamente restrictiva. Se impone un control estricto sobre el entorno, prohibiendo el acceso a internet, el uso de dispositivos USB y, en ocasiones, incluso las visitas a los servicios higiénicos sin permiso. Se obliga a

los estudiantes a utilizar un IDE específico (CLion o NetBeans en Windows), ignorando la diversidad de herramientas y sistemas operativos utilizados en la industria.

Más preocupante es la letanía de restricciones artificiales sobre el propio lenguaje de programación. Se prohíbe explícitamente el uso de bibliotecas estándar de C++ como `<string>`, forzando a los estudiantes a utilizar la gestión manual de cadenas de caracteres con `char*`, una práctica notoriamente insegura y obsoleta. Incluso se prohíbe el uso del carácter de tabulación (`\t`) para formatear la salida, obligando a los estudiantes a realizar cálculos manuales de espaciado, una tarea sin valor pedagógico que solo añade complejidad artificial. El sistema de penalización es desproporcionadamente punitivo hacia los errores formales: un error de sintaxis puede costar el 50% de la nota, mientras que no seguir las convenciones de nomenclatura de carpetas y proyectos puede resultar en la pérdida de hasta 5 puntos sobre 20. Se estima que hasta un 45% de la calificación puede perderse por cuestiones de forma, no de fondo, lo que incentiva la conformidad mecánica por encima de la comprensión y la resolución de problemas.

7.3 Desconexión con la Industria y Obsolescencia Tecnológica

El sistema documentado de la PUCP muestra una alarmante desconexión con la realidad de la industria tecnológica de 2026. La prohibición explícita del uso de “toda forma de Inteligencia Artificial Generativa” en el sílabo de Programación 2, calificando su uso como una “falta a la ética académica”, es un ejemplo flagrante. Herramientas como GitHub Copilot son estándar en la industria y la habilidad para utilizarlas eficazmente es una competencia laboral cada vez más valorada. Prohibirlas es preparar a los estudiantes para un pasado que ya no existe.

La ausencia total de temas como el control de versiones con Git, las pruebas unitarias y de integración (testing), la integración continua (CI/CD), el diseño de APIs y la gestión de dependencias en los sílabos analizados crea una brecha masiva entre la formación recibida y las habilidades requeridas para un puesto de desarrollador de software de nivel de entrada. La entrega de proyectos en archivos ZIP en lugar de a través de repositorios de Git es una práctica que ignora la herramienta de colaboración más fundamental en el desarrollo de software. Este enfoque curricular no solo es obsoleto, sino contraproducente, ya que enseña prácticas que son consideradas anti-patrones en el desarrollo moderno (como la gestión manual de memoria en C++) y omite las habilidades que son absolutamente esenciales para la empleabilidad y la productividad en el sector tecnológico actual.

7.4 Impacto Sistémico: Salud Mental y Viabilidad Económica

La documentación analizada también presenta datos preocupantes sobre el impacto sistémico de este modelo educativo. Se citan estudios que revelan altos niveles de estrés, ansiedad y depresión entre los estudiantes universitarios peruanos, incluyendo los de la PUCP. Un estudio de 2026 indica que el 36% de los universitarios había tenido pensamientos suicidas, una cifra que casi duplica la de 2020. El entorno de alta presión, los plazos ajustados, las evaluaciones constantes y un sistema punitivo que castiga severamente el error, como el descrito en los laboratorios, son identificados como factores contribuyentes a esta crisis de salud mental. Este ambiente, que fomenta el miedo al fracaso y la aversión al riesgo, es antitético al desarrollo de la resiliencia y la audacia necesarias para el emprendimiento.

Desde una perspectiva económica, el informe cuestiona el retorno de la inversión de una educación en este sistema. Con costos que pueden superar los S/ 125,000, una alta probabilidad de “sobreeducación” (graduados en trabajos que no requieren título) y un premium salarial en declive, la propuesta de valor se debilita. Para un aspirante a emprendedor, la inversión financiera y de tiempo (5 a 8 años) en un sistema que induce aversión al riesgo y merma la salud mental parece ser una decisión económicamente irracional. El sistema parece optimizado para producir empleados para roles estables, no para forjar los fundadores innovadores y resilientes que el ecosistema deep-tech demanda.

8. Conclusiones y Síntesis Comparativa

La investigación y el análisis comparativo realizados en este informe dibujan un panorama de dos mundos educativos en marcado contraste. Por un lado, emerge un paradigma de enseñanza de la programación moderno, ágil y alineado con las realidades del siglo XXI. Este modelo, practicado en universidades de élite y promovido por la pedagogía contemporánea, se centra en el estudiante como un agente activo de su propio aprendizaje. Prioriza la comprensión conceptual profunda sobre la memorización superficial, fomenta la resolución de problemas complejos y abiertos, y utiliza la evaluación basada en proyectos y portafolios para medir la competencia práctica. Integra herramientas de vanguardia como la inteligencia artificial y prácticas industriales como el control de versiones y las pruebas automatizadas, preparando a los estudiantes no solo para ser programadores competentes, sino también para ser pensadores críticos, colaboradores eficaces y aprendices de por vida. Este es el entorno que cultiva las habilidades técnicas, la resiliencia y la mentalidad innovadora necesarias para el éxito en el exigente campo del emprendimiento deep-tech.

Por otro lado, el análisis documental del sistema de la Pontificia Universidad Católica del Perú revela un modelo que parece anclado en un pasado pedagógico y tecnológico. Se caracteriza por una estructura rígida y repetitiva, un control autoritario del proceso de aprendizaje y un sistema de evaluación punitivo que valora la conformidad formal por encima de la comprensión funcional. Las restricciones artificiales, como la prohibición de bibliotecas estándar y herramientas de la industria, junto con la omisión de temas fundamentales de la ingeniería de software moderna, crean una brecha significativa entre la formación académica y las demandas del mercado laboral. Este enfoque mecanicista, que castiga el error y desalienta la experimentación, no solo resulta obsoleto, sino que, según los datos presentados, contribuye a un ambiente de alta presión que impacta negativamente en la salud mental de los estudiantes y socava el desarrollo del perfil psicológico requerido para el emprendimiento.

En conclusión, la divergencia entre los dos modelos es fundamental. Mientras que la educación moderna busca empoderar a los estudiantes para que naveguen y den forma a un futuro tecnológico incierto, el sistema tradicional documentado parece diseñado para producir conformidad en un presente que ya ha sido superado. Para una institución que aspira a formar a los líderes tecnológicos del futuro, y para los estudiantes que invierten su tiempo, dinero y bienestar en esa promesa, esta desconexión representa un desafío estructural crítico que exige una reflexión y una reforma profundas. El camino hacia la formación de la próxima generación de innovadores y emprendedores deep-tech no reside en la repetición y la restricción, sino en la autonomía, la experimentación y la conexión auténtica con el dinámico mundo de la tecnología.

References

1. [El aprendizaje basado en problemas: revisión de estudios empíricos internacionales - Revista de Educación - Ministerio de Educación, Formación Profesional y Deportes](https://www.educacionfpydeportes.gob.es/revista-de-educacion/numeros-revista-educacion/numeros-antteriores/2006/re341/re341-17.html) (https://www.educacionfpydeportes.gob.es/revista-de-educacion/numeros-revista-educacion/numeros-antteriores/2006/re341/re341-17.html)
2. (PDF) [El aprendizaje basado en problemas: revisión de estudios empíricos internacionales - ResearchGate](https://www.researchgate.net/publication/28132767_El_aprendizaje_basado_en_problemas_revision_de_estudios_empiricos_internacionales) (https://www.researchgate.net/publication/28132767_El_aprendizaje_basado_en_problemas_revision_de_estudios_empiricos_internacionales)
3. [Aprendizaje basado en problemas y enseñanza por proyectos: alternativas diferentes para enseñar - SciELO](http://scielo.sld.cu/scielo.php?script=sci_arttext&pid=S0257-43142018000100009) (http://scielo.sld.cu/scielo.php?script=sci_arttext&pid=S0257-43142018000100009)
4. [La efectividad del aprendizaje basado en problemas \(abp\) en el desarrollo de la competencia matemática de formulación y resolución de problemas - Repositorio CUC](https://repositorio.cuc.edu.co/entities/publication/61cf095b-84d1-4c78-8118-df3a132270f9) (https://repositorio.cuc.edu.co/entities/publication/61cf095b-84d1-4c78-8118-df3a132270f9)

5. [Aprendizaje Basado en Proyectos: Un enfoque educativo innovador para una enseñanza activa - Reincisol](https://doi.org/10.59282/reincisol.V4(7)320-341) ([https://doi.org/10.59282/reincisol.V4\(7\)320-341](https://doi.org/10.59282/reincisol.V4(7)320-341))
6. [Flipped Classroom, estrategias de aprendizaje y rendimiento en ciencias - academia.edu](https://www.academia.edu/105755911/Flipped_Classroom_estrategias_de_aprendizaje_y_rendimiento_en_ciencias) (https://www.academia.edu/105755911/Flipped_Classroom_estrategias_de_aprendizaje_y_rendimiento_en_ciencias)
7. [El aula invertida en la educación técnica: una revisión sistemática - revistacseducacion.unr.edu.ar](https://revistacseducacion.unr.edu.ar/index.php/educacion/article/view/556) (<https://revistacseducacion.unr.edu.ar/index.php/educacion/article/view/556>)
8. [El aula invertida como metodología innovadora en la educación superior - SciELO Venezuela](https://ve.scielo.org/scielo.php?script=sci_arttext&pid=S2665-02822023000200061) (https://ve.scielo.org/scielo.php?script=sci_arttext&pid=S2665-02822023000200061)
9. [El Flipped Classroom como metodología activa para el aprendizaje de la programación en Educación Secundaria - Titula Universidad Europea](https://titula.universidadeuropea.com/bitstream/handle/20.500.12880/4517/TFM_EstellaLumbrerasMikel.pdf?sequence=1&isAllowed=y) (https://titula.universidadeuropea.com/bitstream/handle/20.500.12880/4517/TFM_EstellaLumbrerasMikel.pdf?sequence=1&isAllowed=y)
10. [Fortalecimiento de las competencias informáticas en estudiantes de educación superior mediante la implementación de la metodología Flipped Classroom en la plataforma Moodle - ojs.southfloridapublishing.com](https://ojs.southfloridapublishing.com/ojs/index.php/jdev/article/view/4281) (<https://ojs.southfloridapublishing.com/ojs/index.php/jdev/article/view/4281>)
11. [La enseñanza programada de B. F. Skinner - Psicología y Mente](https://psicologiaymente.com/develop/ensenanza-programada-skinner) (<https://psicologiaymente.com/develop/ensenanza-programada-skinner>)
12. [La memoria en el aprendizaje: ¿un recurso denostado? - Dialnet](https://dialnet.unirioja.es/descarga/articulo/7539696.pdf) (<https://dialnet.unirioja.es/descarga/articulo/7539696.pdf>)
13. [La programación didáctica: un trabajo en equipo - Redalyc](https://www.redalyc.org/journal/274/27466169007/html/) (<https://www.redalyc.org/journal/274/27466169007/html/>)
14. [Estrategia didáctica para desarrollar la competencia solución de problemas con programación de computadoras - SciELO Perú](http://www.scielo.org.pe/scielo.php?script=sci_arttext&pid=S1019-94032018000200005) (http://www.scielo.org.pe/scielo.php?script=sci_arttext&pid=S1019-94032018000200005)
15. [Modelo de enseñanza tradicional: características y por qué está obsoleto - Psicoblog](https://psicoblog.org/modelo-de-ensenanza-tradicional/) (<https://psicoblog.org/modelo-de-ensenanza-tradicional/>)
16. [Brecha digital y rendimiento académico en estudiantes de computación de una universidad pública mexicana - Acta Universitaria](https://www.actauniversitaria.ugto.mx/index.php/acta/article/view/4517) (<https://www.actauniversitaria.ugto.mx/index.php/acta/article/view/4517>)
17. [INF 111- Informática I - mi.sagrado.edu](https://mi.sagrado.edu/ICS/icsfs/INF_111-_Informa%CC%81tica_I.pdf?target=eba24f60-8af6-4e7e-bb36-4896ef8972c3) (https://mi.sagrado.edu/ICS/icsfs/INF_111-_Informa%CC%81tica_I.pdf?target=eba24f60-8af6-4e7e-bb36-4896ef8972c3)
18. [El exceso de información digital y su influencia en el aprendizaje de los estudiantes de bachillerato - Dialnet](https://dialnet.unirioja.es/servlet/articulo?codigo=8964813) (<https://dialnet.unirioja.es/servlet/articulo?codigo=8964813>)
19. [PDF\) Sistema de Evaluación en Ingeniería del Software - ResearchGate](https://www.researchgate.net/publication/233747025_Sistema_de_Evaluacion_en_Ingenieria_del_Software) (https://www.researchgate.net/publication/233747025_Sistema_de_Evaluacion_en_Ingenieria_del_Software)
20. [Unificando la evaluación de proyectos a través de un software centralizado para la toma de decisiones ágiles en el programa de Ingeniería de Sistemas de la UNAB: Revisión sistemática | Publicaciones e Investigación - UNAD Hemeroteca](https://hemeroteca.unad.edu.co/index.php/publicaciones-e-investigacion/article/view/7699) (<https://hemeroteca.unad.edu.co/index.php/publicaciones-e-investigacion/article/view/7699>)
21. [EVALUACION DE SOFTWARE EDUCATIVO: ORIENTACIONES PARA SU USO PEDAGÓGICO - SKAT IHMC](https://skat.ihmc.us/rid=1NQWB1YN3-83H1W9-4D4Q/lectura27.pdf) (<https://skat.ihmc.us/rid=1NQWB1YN3-83H1W9-4D4Q/lectura27.pdf>)
22. [Redalyc.Revisión de modelos para evaluación de software ... - Redalyc](https://www.redalyc.org/pdf/784/78470106.pdf) (<https://www.redalyc.org/pdf/784/78470106.pdf>)
23. [Gestión de Programas, Portfolios, y Proyectos | ETSISI - ETSISI UPM](https://www.etsisi.upm.es/gestion_programas_portfolios_y_proyectos) (https://www.etsisi.upm.es/gestion_programas_portfolios_y_proyectos)

24. [Introduction to Computer Science and Programming in Python - MIT OpenCourseWare](https://ocw.mit.edu/courses/6-0001-introduction-to-computer-science-and-programming-in-python-fall-2016/) (https://ocw.mit.edu/courses/6-0001-introduction-to-computer-science-and-programming-in-python-fall-2016/)
25. [6.006 Introduction to Algorithms, Fall 2011 - MIT OpenCourseWare](https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/) (https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/)
26. [CS106A: Programming Methodology - Stanford University](https://web.stanford.edu/class/cs106a/) (https://web.stanford.edu/class/cs106a/)
27. [CS 15-122: Principles of Imperative Computation \(Spring 2026\) - Carnegie Mellon University](https://www.cs.cmu.edu/~15122/) (https://www.cs.cmu.edu/~15122/)
28. [CS 61A: Structure and Interpretation of Computer Programs - cs61a.org](https://cs61a.org/) (https://cs61a.org/)
29. [Las tendencias 'deep tech' que marcarán el emprendimiento en 2026 - Expansión](https://www.expansion.com/expansion-empleo/emprendedores/2025/12/29/695243dce5fdeadb708b458f.html) (https://www.expansion.com/expansion-empleo/emprendedores/2025/12/29/695243dce5fdeadb708b458f.html)
30. [Deep Tech Entrepreneurship: Empezar en Sectores de Alta Complejidad - Next IBS](https://www.nextibs.com/deep-tech-entrepreneurship-emprender-sectores-complejidad/) (https://www.nextibs.com/deep-tech-entrepreneurship-emprender-sectores-complejidad/)
31. [Las habilidades tech más demandadas: qué aprender para no quedarte atrás - SoyHenry Blog](https://blog.soyhenry.com/las-habilidades-tech-mas-demandadas-que-aprender-para-no-quedarte-atras/) (https://blog.soyhenry.com/las-habilidades-tech-mas-demandadas-que-aprender-para-no-quedarte-atras/)
32. [EmprendeTech - IESA](https://www.iesa.edu.ve/cursos-y-programas/cursos/emprendetech) (https://www.iesa.edu.ve/cursos-y-programas/cursos/emprendetech)
33. [Habilidades blandas, clave para el éxito profesional - AL DÍA - Universidad Iberoamericana](https://aldia.iberomx/content.php?Division=DEC&pageId=1900) (https://aldia.iberomx/content.php?Division=DEC&pageId=1900)
34. [Dentro del 'boom loop' de Y Combinator: la fábrica de 'startups' se hace más fuerte, más esbelta y más malvada - Forbes España](https://forbes.es/start-ups/424003/dentro-del-boom-loop-de-y-combinator-la-fabrica-de-startups-se-hace-mas-fuerte-mas-esbelta-y-mas-malvada/) (https://forbes.es/start-ups/424003/dentro-del-boom-loop-de-y-combinator-la-fabrica-de-startups-se-hace-mas-fuerte-mas-esbelta-y-mas-malvada/)
35. [¿Qué es Y Combinator? - doola](https://www.doola.com/es/blog/what-is-y-combinator/) (https://www.doola.com/es/blog/what-is-y-combinator/)
36. [El hombre que susurraba a las 'start-ups' - MIT Technology Review](https://technologyreview.es/article/business-impact-el-hombre-que-susurraba-las-start-ups/) (https://technologyreview.es/article/business-impact-el-hombre-que-susurraba-las-start-ups/)
37. [Y Combinator: La aceleradora de startups más importante del mundo - Ecosistema Startup](https://ecosistemastartup.com/actores/y-combinator/) (https://ecosistemastartup.com/actores/y-combinator/)
38. [Y Combinator: 2 fundadores de startups comparten sus consejos para ser elegidos por la aceleradora más importante del mundo - Business Insider España](https://www.businessinsider.es/tecnologia/combinator-fundadores-startups-comparten-consejos-ser-elegidos-1120907) (https://www.businessinsider.es/tecnologia/combinator-fundadores-startups-comparten-consejos-ser-elegidos-1120907)
39. [¿Por qué elegir GitHub Copilot como herramienta de Inteligencia Artificial para programadores? - ID Bootcamps](https://iddigitalschool.com/bootcamps/por-que-elegir-github-copilot-como-herramienta-de-inteligencia-artificial-para-programadores/) (https://iddigitalschool.com/bootcamps/por-que-elegir-github-copilot-como-herramienta-de-inteligencia-artificial-para-programadores/)
40. [Curso de GitHub Copilot: programación con IA - ADR Formación](https://www.adrformacion.com/curso-online/github-copilot-programacion-inteligencia-artificial) (https://www.adrformacion.com/curso-online/github-copilot-programacion-inteligencia-artificial)
41. [El impacto de herramientas de programación inteligente en ingeniería: Evaluando el uso de Github Copilot - Helvia - Universidad de Córdoba](https://helvia.uco.es/xmlui/handle/10396/32397) (https://helvia.uco.es/xmlui/handle/10396/32397)
42. [Curso gratis de GitHub Copilot para programar más rápido con IA - CursotecaPlus](https://cursotecaplus.com/curso-gratis-de-github-copilot-para-programar-mas-rapido-con-ia/) (https://cursotecaplus.com/curso-gratis-de-github-copilot-para-programar-mas-rapido-con-ia/)
43. [GitHub Copilot · Your AI pair programmer - GitHub](https://github.com/features/copilot) (https://github.com/features/copilot)
44. [Soft skills más demandadas este 2026 - Trabajos.com](https://blog.trabajos.com/2025/12/10/soft-skills-mas-demandadas/) (https://blog.trabajos.com/2025/12/10/soft-skills-mas-demandadas/)
45. [Las soft skills más demandadas por las empresas en 2025 - Elena Arnaiz](https://elenaarnaiz.es/soft-skills-mas-demandadas-por-las-empresas/) (https://elenaarnaiz.es/soft-skills-mas-demandadas-por-las-empresas/)
46. [Las 10 habilidades tecnológicas más demandadas para el 2024 - IEBS Business School](https://www.iebschool.com/hub/tech-skills-mas-demandadas/) (https://www.iebschool.com/hub/tech-skills-mas-demandadas/)

47. [Las 10 soft skills más demandadas por las empresas en 2025 - Rey Ardid](https://www.reyardid.org/blog/soft-skills-mas-demandadas/) (<https://www.reyardid.org/blog/soft-skills-mas-demandadas/>)